# ONNX Passes: Graph Transformation Infrastructure based on ONNX Script

**Christoph Berganski**
Computer Engineering Group
Paderborn University

SAIL
SUSTAINABLE LIFE-CYCLE OF INTELLIGENT SOCIO-TECHNICAL SYSTEMS

Ministerium für Kultur und Wissenschaft des Landes Nordrhein-Westfalen

PADERBORN UNIVERSITY

eki
ENERGIEEFFIZIENTE KI DURCH DNN-APPROXIMATION FÜR FPGAs

Supported by:
Federal Ministry for the Environment, Nature Conservation, Nuclear Safety and Consumer Protection
based on a decision of the German Bundestag

## ONNX IR and ONNX Script

Infrastructure developed by **ONNX** and **Microsoft**

ONNX IR-Py

*Efficient in-memory representation for ONNX, in Python*

- ❖ Faster, more accessible and readable compared to raw ONNX/protobuf
- ❖ Base classes for passes and basic cleanup and validation passes

ONNX SCRIPT

*... naturally author ONNX functions and models using a subset of Python*

- ❖ Base classes for pattern-based rewrite rules and ONNX graph builder
- ❖ Scripted functions for custom operators and test models

## Tightly Verified Transformations

Execute the model after **each pass** (that modified the model) via the ONNX Reference Evaluator to verify correctness.

Easily enabled by inheriting the Verify base and optionally annotating with @tolerance or @metric for tolerance or metric-based (e.g. accuracy) verification.

```
@equality  # @equality is the default
class Mistake_v1(RewriteRule, Verify):
    """Makes a mistake."""
```

```
@tolerance
class Mistake_v1(RewriteRule, Verify):
    """Makes a mistake."""
```

```
@metric
class Mistake_v1(RewriteRule, Verify):
    """Makes a mistake."""
```

Raises exception when model outputs do not match exactly, within tolerance, or cause a metric degradation (also logged when accepted).

## Pattern-based Rewrite Rules

Disentangle (a) detecting a pattern, (b) generating/inserting a transformed pattern and (c) conditions under which a transformation should/can be applied

Instantiate a RewriteRule by filling in the pattern, check and rewrite methods:

- ❖ ONNX Script graph builder constructs pattern and rewrite
- ❖ Access to matched values and context with model and metadata in check
- ❖ Mix with Python and NumPy in check and rewrite to calculate values

```
@tolerance
class RewriteSubAsAdd_v1(RewriteRule, Verify):
    """Rewrite subtraction as addition of the negative."""

    @staticmethod
    def pattern(op, x, y):
        return op.Sub(x, y)

    @staticmethod
    def check(context, x, y):
        return y.dtype is not None and y.dtype.is_signed()

    @staticmethod
    def rewrite(op, x, y):
        return op.Add(x, op.Neg(y))
```

ONNX Script manages pattern matching, extraction and rewiring the graph

## Generic Patterns

Reordering and elimination passes as instances of **algebraic properties** making use of **templates** and **inheritance** where possible

*associativity, commutativity, distributivity, identity, inverse, involution, idempotence, complement, annihilator, absorption, etc.*

- ❖ Fill in a single operator placeholder and the constant identity element
- ❖ Match condition check and rewrite are implemented in the base class
- ❖ Tag as commute to not spell out both variations x + 0 and 0 + x explicitly

```
class EliminateIdentityAdd_v1(EliminateIdentity, Verify):
    """Eliminate addition of the identity element."""

    identity = 0

    @staticmethod
    def operator(op, x, y, *args, **kwargs):
        return op.Add(x, y, *args, **kwargs)

    @property
    def commute(self) -> bool:
        return True
```

```
class EliminateAbsorption_v1(RewriteRuleSetTemplate, Verify):
    """Eliminate absorption identity x . (x * y) -> x."""

    patterns = (
        lambda op: (op.And, op.Or),
        lambda op: (op.Or, op.And),
        lambda op: (op.Min, op.Max),
        lambda op: (op.Max, op.Min),
    )

    # Once adding non-commutative operations, up
    # to four different orders need to be spelled out:
    #   x . (x * y) -> x   (1)
    #   x . (y * x) -> x   (2)
    #   (x * y) . x -> x   (3)
    #   (y * x) . x -> x   (4)
    @property
    def commute(self) -> bool:
        return True

    @staticmethod
    def pattern(partial, op, x, y):
        return partial(op)[0](x, partial(op)[1](x, y))

    @staticmethod
    def rewrite(partial, op, x, y):
        return op.Expand(x, op.Shape(partial(op)[1](x, y)))
```

- ❖ List partial patterns inserted into placeholders to generate a rule set
- ❖ Tag as commute to not spell out all pattern variations explicitly
- ❖ A pattern and rewrite template with placeholders for partial patterns

## Custom Operators via Scripted Functions

Model custom and auxiliary operations as part of the ONNX graph via ONNX Script functions: **Enables end-to-end execution and shape inference**

```
class MultiThreshold_v1(OnnxOperator):
    """Multi-Threshold operator for quantized activations and layer tails.
    """

    @staticmethod
    def script(op: Opset):
        """Generate an ONNX Script function implementing MultiThreshold."""

        def multithreshold(x, thresholds, weights):
            # Comparison of inputs and all corresponding thresholds: Expand
            # input dimensions to match the threshold parameter shape via
            # broadcasting
            steps = op.GreaterOrEqual(op.Unsqueeze(x, axes=[-1]), thresholds)
            # Type-casting turns boolean unit steps to reducible floats
            steps = op.Cast(steps, to=ir.DataType.FLOAT)
            # Finally the multi-threshold output reduces over all steps removing
            # the previously expanded dimension
            return op.ReduceSum(weights * steps, [-1], keepdims=0)

        return multithreshold
```

- ❖ Registry and versioning of custom operators
- ❖ Python function describes the interface: inputs
- ❖ Custom behavior in terms of standard operations
- ❖ Python return describes the operator output

## Design Principles

**Comply with the ONNX Standard/Specification**
Don't break the graph! Not even temporarily.

**Avoid model-specific ad-hoc solutions**
A library of baseline transformations applicable to any model?

**Follow a bottom-up approach**
Compose complex transformations from small, reusable and generic patterns

## ONNX Passes on GitHub

Ongoing effort to create an optimizing compiler frontend for custom ONNX compilers

Applicable to FINN and beyond

Platform for my personal research on model graph optimization and compression
*(happy to discuss more esoteric/crazy ideas...)*

SCAN ME